

Costruttori e visibilita

Costruttori e visibilita

Come inizializzare oggetti e controllare l'accesso a proprieta e metodi.

Il costruttore

Il costruttore e un metodo speciale chiamato automaticamente quando crei un oggetto.

```
<?php
class Prodotto {
    public string $nome;
    public float $prezzo;

    public function __construct(string $nome, float $prezzo) {
        $this->nome = $nome;
        $this->prezzo = $prezzo;
    }
}

$prodotto = new Prodotto("Quaderno", 3.50);
```

`__construct` prepara l'oggetto con i dati iniziali.

Visibilita

La visibilita decide da dove puoi usare proprieta e metodi.

- `public` : accessibile anche dall'esterno
- `private` : accessibile solo dentro la classe
- `protected` : accessibile dentro la classe e nelle classi figlie

Private

```
<?php
class Conto {
    private float $saldo = 0;

    public function deposita(float $importo): void {
        if ($importo > 0) {
            $this->saldo += $importo;
        }
    }

    public function leggiSaldo(): float {
        return $this->saldo;
    }
}
```

`$saldo` è privato: dall'esterno non puoi modificarlo direttamente. Devi passare dai metodi.

Incapsulamento

L'incapsulamento significa proteggere i dati interni di un oggetto e offrire metodi chiari per usarli.

Questo aiuta a evitare valori impossibili, come un deposito negativo.

Getter e setter

Un **getter** legge un valore. Un **setter** lo modifica controllando prima se va bene.

```
public function leggiSaldo(): float {
    return $this->saldo;
}
```

Non servono sempre, ma sono utili quando vuoi controllare l'accesso ai dati.

Promozione delle proprietà nel costruttore

In PHP moderno puoi scrivere costruttori più compatti.

```
<?php
class Prodotto {
    public function __construct(
        public string $nome,
        private float $prezzo
    ) {
    }
}
```

Questa sintassi crea le proprietà e le valorizza nello stesso punto. È comoda, ma all'inizio va letta con calma:

`public string $nome` dentro il costruttore diventa una proprietà pubblica della classe.

Setter con controllo

Un setter serve quando vuoi permettere una modifica, ma solo se il valore è valido.

```
<?php
class Prodotto {
    private float $prezzo;

    public function cambiaPrezzo(float $prezzo): void {
        if ($prezzo <= 0) {
            return;
        }

        $this->prezzo = $prezzo;
    }
}
```

Qui impedisce prezzi uguali o minori di zero.

Regola pratica

Usa `private` per i dati che non devono essere modificati liberamente. Offri metodi pubblici quando vuoi controllare come quei dati vengono letti o cambiati.